

BigDecimal v1.2 Add-in

Functional Understanding Document

Version 1.1

Table of Contents

1	Introduction	4
1.1	Why BigDecimal?	4
2	Installation of BigDecimal v1.2.xlam	4
2.1.1	Downloading the BigDecimal Add-in	4
2.1.2	Important: Unblocking the Add-in	5
3	Working with BigDecimal	6
3.1	Referencing BigDecimal in VBA	6
4	BigDecimal "Quick Start" Guide	6
4.1	Instantiating the BigDecimal Class	7
4.2	Casting standard types	7
4.3	Implementation Example: Basic Addition	7
4.4	Language Limitations: Operator Overloading	8
5	Functional Specifications	8
5.1	BigDecimal Methods	9
5.2	BigDecimal Properties	9
5.2.1	The Default .Value Property	10
5.2.2	The .ValueEx Property	10
5.2.3	The .MaxScale Property	11
5.2.4	Auto-Casting of Standard Types	12
5.2.5	Effortless Precision with Auto-Trimming	13
5.2.6	Mutable vs. Immutable Properties	13
5.3	Global Configuration with BigOption	13
5.4	Active State Management	14
6	Conclusion	15

List of Tables

Table 5-1: BigDecimal Methods	9
Table 5-2: BigDecimal Properties	10
Table 5-3: .ValueEx Property VBA Types.....	11
Table 5-4: BigOption Properties	14

List of Figures

Figure 2-1: Excel Add-ins Folder.....	5
Figure 2-2: Unblock Downloaded BigDecimal Add-in.....	5
Figure 3-1: Adding BigDecimal_v1_2 as a Reference	6

1 Introduction

Welcome to BigDecimal, a high-performance, arbitrary-precision decimal library designed to break the 15-digit precision barrier of standard Excel calculations. Heavily inspired by the robust BigDecimal implementations found in enterprise languages like Java, this add-in brings industrial-grade numerical accuracy directly to your spreadsheets.

Whether you are performing complex financial reconciliations or deep scientific research, BigDecimal ensures that your results are never compromised by floating-point errors or rounding limitations.

1.1 Why BigDecimal?

Standard Excel uses the IEEE 754 floating-point standard, which is excellent for speed but can introduce "silent" errors in very large or very small numbers. BigDecimal for Excel replaces this with an exact-value representation, allowing you to define the exact scale and precision required for your specific use case.

2 Installation of BigDecimal v1.2.xlam

Follow the steps below to install the **BigDecimal** add-in and integrate it with Microsoft Excel. These steps ensure the library is correctly registered and available for use within the VBA Editor.

2.1.1 Downloading the BigDecimal Add-in

To install the Add-in, download the **BigDecimal v1.2.xlam** file from the [GitHub](#) repository and move it to the local Microsoft Add-ins directory:

```
C:\Users\[YourUsername]\AppData\Roaming\Microsoft\AddIns
```

Note: Replace [YourUsername] with your active Windows login name.

Quick Access Shortcut:

Alternatively, you can access this folder directly by pasting the following environment variable into the File Explorer address bar:

```
%appdata%\Microsoft\AddIns
```

Once the folder is open, drag and drop the BigDecimal v1.2.xlam file from your **Downloads** folder into the **Addins** folder as in Figure 2-1.

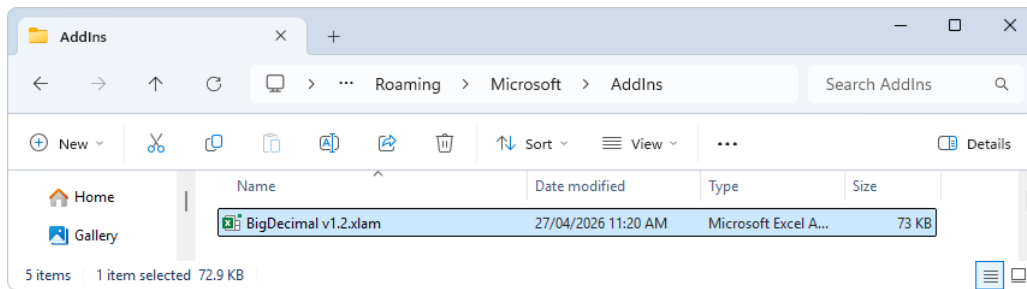


Figure 2-1: Excel Add-ins Folder

2.1.2 Important: Unblocking the Add-in

Because the file was downloaded from an external source (GitHub), Windows will apply a 'Mark of the Web' security attribute, which prevents the Add-in from executing macros. Even if the file is placed in the correct directory, it will not function until it is manually unblocked.

To unblock the Add-in:

1. Navigate to the %appdata%\Microsoft\AddIns folder
2. **Right-click** the BigDecimal v1.2.xlam file.
3. Select **Properties**.
4. At the bottom of the **General** tab, look for **Security**.
5. Check the **Unblock** box and click **Apply**.

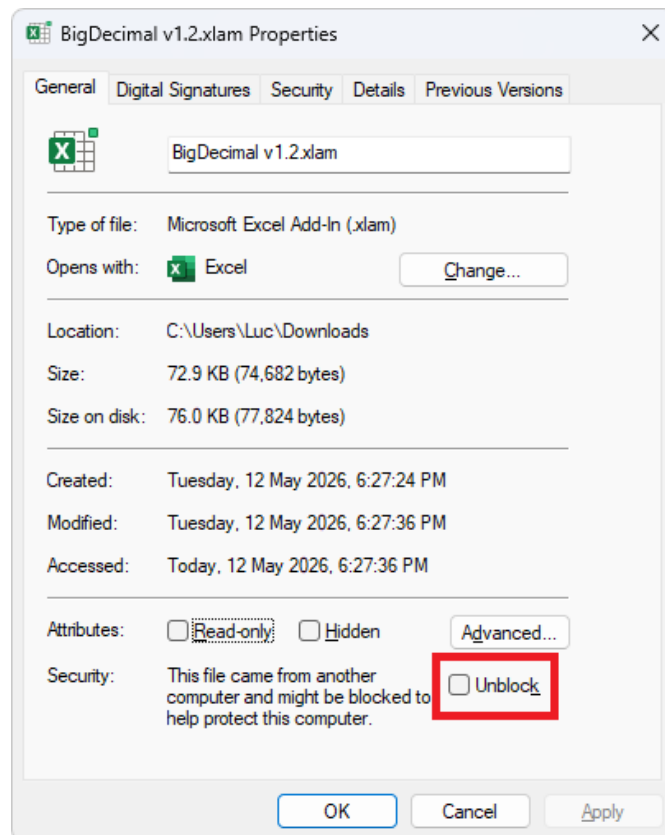


Figure 2-2: Unblock Downloaded BigDecimal Add-in

3 Working with BigDecimal

Before the **BigDecimal** class can be utilized in your VBA project, it must be properly initialized within the VBA environment. This involves linking the Add-in to your specific workbook and ensuring the VBA compiler can access the library's properties and methods.

3.1 Referencing BigDecimal in VBA

Once the Add-in is installed and unblocked, you must establish a reference to the library within your specific VBA Project. This allows the VBA compiler to recognize the **BigDecimal** class and provide IntelliSense support while coding.

Follow these steps to enable the reference:

1. Open Excel.
2. Go to the **Developer** tab (If you don't see it, right-click any ribbon tab > Customize the Ribbon > Check "Developer").
3. Click **View Code**.
4. Go to the **Tools** menu > **References..** and locate BigDecimal_v1_2 and select it as in Figure 3-1. A tick mark will appear next to BigDecimal_v1_2.
5. Click **OK**.

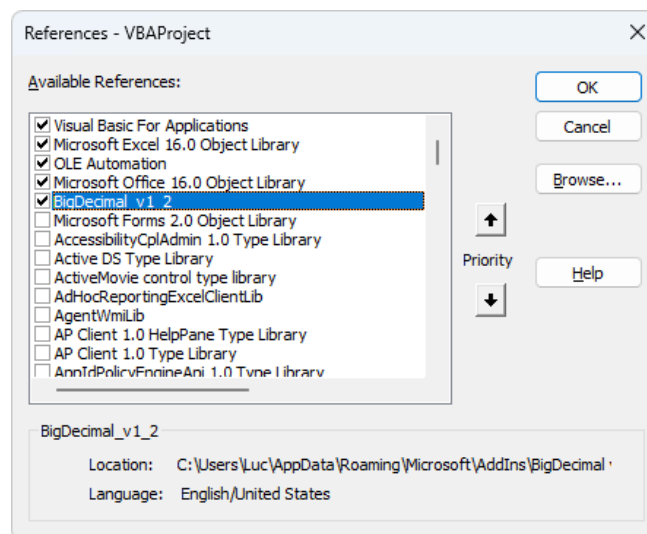


Figure 3-1: Adding BigDecimal_v1_2 as a Reference

At this stage the add-in is now available to use in your VBA Project.

4 BigDecimal “Quick Start” Guide

BigDecimal is designed to work seamlessly within the Excel environment, its implementation respects the standard VBA architectural patterns used by Microsoft Office. To ensure maximum compatibility and performance across different workbooks, the library utilizes a

controlled instantiation model. This approach prevents common reference errors and ensures that every `BigDecimal` object is correctly initialized with the high-precision engine.

4.1 Instantiating the `BigDecimal` Class

In standard object-oriented programming, you would typically instantiate a class using the `New` keyword. However, due to the **PublicNotCreatable** nature of classes within an Excel Add-in environment, the `BigDecimal` class cannot be created directly by external projects.

To bypass this limitation, the Add-in provides a global factory function: `New_BigDecimal()`. This function serves as the entry point for object creation, allowing you to initialize a new `BigDecimal` instance. It optionally accepts a string parameter to set the initial value of the number, ensuring precision is maintained from the moment of instantiation.

```
Public Function New_BigDecimal(Optional StrValue As String = "") As  
    BigDecimal
```

4.2 Casting standard types

To simplify workflows, the Add-in includes the `CBgDec()` casting function. This function mirrors native VBA conversion functions (like `CDBl` or `CDec`), accepting standard data types and returning a fully initialized `BigDecimal` object.

```
Public Function CBgDec(Value As Variant) As BigDecimal
```

Supported Conversions

The `CBgDec()` function safely handles the following native types by converting them to their string representations internally before initializing the `BigDecimal` object:

- **Numeric Types:** Integer, Long, LongLong, Single, Double.
- **Strings:** Standard text representations of numbers (e.g., "123.4567890123456789").

4.3 Implementation Example: Basic Addition

The following example, adapted from the **BigCalculator** source code, demonstrates how to instantiate two `BigDecimal` objects from user input, perform basic validation, and output the result of an addition operation.

```

Private Sub Add_Click()
    Dim Num1 As BigDecimal
    Dim Num2 As BigDecimal

    With BigDecimalForm
        ' Instantiate objects using the New_BigDecimal factory function
        Set Num1 = New_BigDecimal(.Num1.Value)
        Set Num2 = New_BigDecimal(.Num2.Value)

        ' Validate inputs and check for successful instantiation
        If .Num1.Value = "" Then
            ErrBox "Number 1 is empty!", "Input Error"
            Exit Sub
        ElseIf .Num2.Value = "" Then
            ErrBox "Number 2 is empty!", "Input Error"
            Exit Sub
        ElseIf Num1 Is Nothing Then
            ErrBox "Number 1 is invalid!", "Validation Error"
            Exit Sub
        ElseIf Num2 Is Nothing Then
            ErrBox "Number 2 is invalid!", "Validation Error"
            Exit Sub
        End If

        ' Perform addition and return the result as a string
        .Answer.Value = Num1.Plus(Num2).ValueEx(vbString)
    End With
End Sub

```

4.4 Language Limitations: Operator Overloading

A notable constraint of the VBA environment is the lack of **Operator Overloading**. In many modern languages, you can redefine standard operators (such as +, -, *, or /) to work directly with custom classes. Ideally, one would use a syntax like `Set Result = Num1 + Num2`; however, this is not supported in VBA.

Consequently, there is no direct workaround for this language limitation. All arithmetic operations must be performed by calling the explicit methods of the `BigDecimal` class. While this requires a slightly more verbose coding style, the method names are intuitive and follow standard mathematical naming conventions.

5 Functional Specifications

This section provides a detailed technical breakdown of the public classes, methods, and properties that comprise the **BigDecimal** framework. These components form the core engine of the library, enabling the programmatic initialization of high-precision variables, the execution of complex mathematical operations, and the fine-tuning of calculation parameters such as scale and rounding. By leveraging these members, developers can extend the numerical capabilities of their Excel applications far beyond the standard limits of floating-point arithmetic.

5.1 BigDecimal Methods

The following table outlines the comprehensive suite of arithmetic and utility methods supported by the **BigDecimal** class, designed to provide consistent results for high-precision computational tasks.

Methods	VBA Syntax (BigDecimal)	Notes
Plus	Num1.Plus(Num2)	Returns the sum of Num1 and Num2.
Minus	Num1.Minus(Num2)	Returns the difference (Num1 - Num2).
Times	Num1.Times(Num2)	Returns the product of Num1 and Num2.
Divide	Num1.Divide(Num2)	Returns the quotient; precision depends on settings.
Remdr	Num1.Remdr(Num2)	Returns the remainder of the division.
Pow	Num1.Pow(Num2)	Calculates the power (x^y)
Sqrt	Num1.Sqrt()	Calculates the square root to current precision.
Ln	Num1.Ln()	Calculates the Natural Logarithm ($\ln$).
Exp	Num1.Exp()	Calculates the Natural Exponent (e^x).
Fact	Num1.Fact()	Calculates the product of all positive integers.
AbsVal	Num1.AbsVal()	Returns the absolute (non-negative) value.
Neg	Num1.Neg()	Returns the value multiplied by -1.
Trunc	Num1.Trunc()	Returns the integer part without rounding.
Fract	Num1.Fract()	Returns only the decimal/fractional part.
Round	Num1.Round(Scale)	Rounds to a specified number of decimal places.
Trim	Num1.Trim()	Trim leading and trailing 0
Cmp	Num1.Cmp(Num2)	Returns -1 (less), 0 (equal), or 1 (greater).

Table 5-1: BigDecimal Methods

5.2 BigDecimal Properties

The **BigDecimal** class includes a set of public properties that provide critical insights into the state and value of your numerical objects. Unlike standard numeric types in VBA, which require complex parsing to analyse, these properties allow you to directly access the internal characteristics of a number—such as its string representation, and its current scale (the number of digits after the decimal point). By utilizing these properties, developers can easily integrate high-precision results into the Excel UI or use them as conditional logic in complex financial and scientific algorithms.

Property	Access	Notes
.Value	Read/Write	Gets or sets the BigDecimal object itself. This is the default property called when no property is specified.
.ValueEx	Read Only	Gets the value cast to a vbInteger, vbLong, vbLongLong, vbSingle, vbDouble, vbString.
.MaxScale	Read/Write	Gets or sets the number of decimal places for a BigDecimal number. Setting this can truncate or pad the number. Default is 100 decimal places.
.AutoRound	Read/Write	A boolean flag that determines if the BigDecimal number should be automatically rounded. Default is True.
.OutputDebug	Read/Write	Enables internal logging to the VBA Immediate Window (Ctrl+G). Default is False.
.Pi	Read Only	Returns the constant Pi as a high-precision BigDecimal object.

Table 5-2: BigDecimal Properties

5.2.1 The Default .Value Property

The .Value property is configured as the **Default Member** of the BigDecimal class. This means the property is automatically invoked when the object is referenced without an explicit property call. This design pattern allows for cleaner, more intuitive code that mimics the behaviour of native VBA variables.

```
' Instead of explicit property calls:
Num1.Value = Num2.Value

' You can simply write:
Num1 = Num2
```

In the example above, the default property mechanism handles the transfer of data: Num2 yields its value, and Num1 accepts it. This syntactic sugar is particularly powerful when passing BigDecimal objects into standard Excel functions or performing quick assignments within loops.

5.2.2 The .ValueEx Property

While the default .Value property is ideal for general assignments, the .ValueEx property provides a typed interface for extracting data. This property accepts a single parameter—a standard VBA VbVarType constant—to determine the format of the return value.

Syntax

```
result = BigDecimal.ValueEx(ReturnType)
```

Supported Return Types

Constant	VBA Type	Use Case
<code>vbInteger</code>	Integer	Small whole numbers (-32,768 to 32,767).
<code>vbLong</code>	Long	Standard 32-bit integer operations.
<code>vbLongLong</code>	LongLong	64-bit integer operations (Available in 64-bit Office).
<code>vbSingle</code>	Single	Low-precision floating point.
<code>vbDouble</code>	Double	Standard Excel cell values and math functions.
<code>vbString</code>	String	Maintaining full precision for display or storage.

Table 5-3: .ValueEx Property VBA Types

Example Usage

```
Dim MyStr As String
Dim MyDb1 As Double

' Retrieve as a string to preserve every decimal place
MyStr = Num1.ValueEx(vbString)

' Retrieve as a double for use in standard VBA math
MyDb1 = Num1.ValueEx(vbDouble)
```

5.2.3 The .MaxScale Property

The .MaxScale property determines the maximum number of digits allowed after the decimal point. This property is crucial for managing computational overhead and ensuring that operations like division—which can result in infinite repeating decimals—have a defined termination point.

Unlike standard VBA types that have a fixed precision, `BigDecimal` allows you to expand or contract this scale dynamically.

Usage and Behavior

- **Expansion:** Increasing .MaxScale will effectively "pad" the number with trailing zeros, preparing it for higher-precision arithmetic.
- **Contraction:** Decreasing .MaxScale will trigger a precision adjustment. Depending on the internal logic, the number will be truncated or rounded to fit the new scale.
- **Read/Write:** This property can be checked to verify current precision or set to enforce a specific scale before a calculation.

Example: Controlling Precision

```
Dim Num1 As New BigDecimal
Num1 = "10.5555"

' Set the scale to 2 decimal places
Num1.MaxScale = 2

Debug.Print Num1.ValueEx(vbString) ' Output: 10.55 (truncation)

' Increase scale for scientific precision
Num1.MaxScale = 10
Debug.Print Num1.ValueEx(vbString) ' Output: 10.55
```

Performance Tip: While `BigDecimal` can handle very high scales, setting an unnecessarily large `.MaxScale` (e.g., 1,000+) for simple calculations may impact performance. It is best practice to set the scale to the minimum required for your specific use case.

5.2.4 Auto-Casting of Standard Types

To further enhance usability, the `BigDecimal` class supports **Auto-Casting** (Implicit Conversion). When a standard VBA numeric type or string is passed to a `BigDecimal` method (see Table 5-2) or assignment, the class automatically converts the input into a high-precision object before processing.

This eliminates the need to manually wrap literal values or standard variables, making arithmetic operations feel natural and concise.

Supported Types for Auto-Casting:

- **Integers:** `vbInteger`, `vbLong`, `vbLongLong`
- **Floating Point:** `vbSingle`, `vbDouble`
- **Strings:** `vbString` (Parsed directly into the internal format)

Example: Simplified Arithmetic

```
' Without auto-casting, you would have to write:
' Num1 = Num1.Add(New_BigDecimal("1"))

' With auto-casting, standard types are handled instantly:
Num1 = Num1.Add(1)           ' Long is auto-cast
Num1 = Num1.Multiply(0.5)    ' Double is auto-cast
Num1 = "100.50"              ' String is auto-cast via the default
                              .Value property
```

Developer Note: Auto-casting ensures that even when interacting with standard VBA types, the precision of the existing `BigDecimal` object remains the priority, preventing accidental "precision leakage" from the standard `Double` type until the operation is complete.

Precision Warning: When using `.ValueEx(vbDouble)` or `.ValueEx(vbSingle)`, values may be rounded or truncated to fit the limitations of the native VBA type. For values exceeding the range of a Long or Double, an overflow error may occur unless handled within your implementation.

5.2.5 Effortless Precision with Auto-Trimming

The **BigDecimal** class is designed to deliver clean, professional output without the need for manual string manipulation. While the internal object may retain leading and trailing zeros to preserve the integrity of its data structure, the **.ValueEx(vbString)** property handles the cleanup for you. Every time you access the value via this property, the library automatically trims any non-significant zeros, ensuring your results are presented in their most concise and readable form while maintaining the high-precision accuracy you require.

5.2.6 Mutable vs. Immutable Properties

Most properties in the **BigDecimal** class are **Mutable**, meaning you can change the value of an existing object without needing to create a new one using the factory function.

- **Direct Modification:** By setting `.StrValue` or `.DbfValue`, you can reuse an existing **BigDecimal** variable and update its internal data instantly.
- **Precision Control:** Manually setting the `.Scale` allows you to force a specific decimal length on an existing number, which is useful for formatting financial data (e.g., forcing 2 decimal places).
- **The Pi Exception:** Unlike the other properties, **Pi** is a constant. It cannot be overwritten, as it represents a fundamental mathematical value. Attempting to set `Num1.Pi = 3` will result in a compilation error, ensuring the integrity of your geometric calculations.

5.3 Global Configuration with BigOption

The **BigOption** class provides a centralized mechanism for managing the behavior of all **BigDecimal** objects within your application. By setting properties on this global object, you define the default environment for any new **BigDecimal** instantiated thereafter.

This "seamless management" approach ensures that you don't have to manually configure every variable, reducing boilerplate code and preventing precision mismatches in large-scale projects.

Supported Global Options

Property	Description
.MaxScale	Sets the default number of decimal places for all newly created BigDecimal objects. Default is 100.
.AutoRound	A boolean flag that determines if numbers should be automatically rounded (e.g., Round Half to Even) or truncated when they exceed the MaxScale. Default is True.
.OutputDebug	Enables internal logging to the VBA Immediate Window (Ctrl+G). Useful for tracing complex calculation steps or identifying performance bottlenecks. Default is False.

Table 5-4: BigOption Properties

5.4 Active State Management

The **BigOption** class is not merely a collection of defaults; it is a **Live Manager**. Using an internal dictionary, it maintains a registry of every active BigDecimal instance within the project. This allows for dynamic, real-time updates across the entire application state.

Key Mechanisms:

- **Global Synchronization:** When you change a property on BigOption (such as .MaxScale), the change is instantly propagated to every existing BigDecimal object currently in memory.
- **Lifecycle Awareness:** The manager intelligently tracks the lifecycle of your objects. When a BigDecimal is terminated or set to Nothing, BigOption removes it from the registry to prevent memory leaks and ensure that only valid, active objects receive updates.
- **Real-Time Control:** This architecture allows you to change the behaviour of your algorithms on the fly. For example, you can increase the precision of all active variables right before a sensitive calculation and lower it immediately afterward to optimize performance.

Example: Real-Time Global Updates

```
Dim Num1 As New BigDecimal, Num2 As New BigDecimal
Num1 = 1.12345
Num2 = 9.87654

' Both Num1 and Num2 are currently at default MaxScale (e.g., 20)
Debug.Print Num1.ValueEx(vbString) ' Outputs 1.123450...

' Update every active BigDecimal in the project instantly
BigOption.MaxScale = 2

Debug.Print Num1.ValueEx(vbString) ' Now outputs 1.12
Debug.Print Num2.ValueEx(vbString) ' Now outputs 9.87
```

Advanced Note: Because BigOption manages the collection via a dictionary, the "Auto-Casting" and "Default Value" features remain lightning-fast. The manager ensures that the overhead of tracking objects does not interfere with the high-speed arithmetic processing of the individual classes.

6 Conclusion

Thank you for choosing **BigDecimal**.

The goal of this library is to provide a robust, high-precision numerical engine that bridges the gap between standard VBA data types and the rigorous accuracy required for modern financial and scientific applications.

I hope the current feature set—from the intelligent **auto-trimming** logic to the definitive precision control of **MaxScale**—empowers you to build more reliable and professional tools. As the development of this class continues, I am committed to maintaining this balance of mathematical integrity and developer-friendly flexibility.